JavaScript is disabled on your browser.

compact1, compact2, compact3

java.util

Class EnumSet<E extends Enum<E>>

java.lang.Object
    java.util.AbstractCollection<E>
        java.util.AbstractSet<E>
            java.util.EnumSet<E>

All Implemented Interfaces:

Serializable, Cloneable, Iterable<E>, Collection<E>, Set<E>

```
public abstract class EnumSet<E extends Enum<E>>
extends AbstractSet<E>
implements Cloneable, Serializable
```

A specialized Set implementation for use with enum types. All of the elements in an enum set must come from a single enum type that is specified, explicitly or implicitly, when the set is created. Enum sets are represented internally as bit vectors. This representation is extremely compact and efficient. The space and time performance of this class should be good enough to allow its use as a high-quality, typesafe alternative to traditional `int`-based "bit flags." Even bulk operations (such as `containsAll` and `retainAll`) should run very quickly if their argument is also an enum set.

The iterator returned by the `iterator` method traverses the elements in their *natural order* (the order in which the enum constants are declared). The returned iterator is *weakly consistent*: it will never throw ConcurrentModificationException and it may or may not show the effects of any modifications to the set that occur while the iteration is in progress.

Null elements are not permitted. Attempts to insert a null element will throw NullPointerException. Attempts to test for the presence of a null element or to remove one will, however, function properly.

Like most collection implementations, `EnumSet` is not synchronized. If multiple threads access an enum set concurrently, and at least one of the threads modifies

the set, it should be synchronized externally. This is typically accomplished by synchronizing on some object that naturally encapsulates the enum set. If no such object exists, the set should be "wrapped" using the `Collections.synchronizedSet(java.util.Set<T>)` method. This is best done at creation time, to prevent accidental unsynchronized access:

```
Set<MyEnum> s = Collections.synchronizedSet(EnumSet.noneOf(MyEnum.class));
```

Implementation note: All basic operations execute in constant time. They are likely (though not guaranteed) to be much faster than their `HashSet` counterparts. Even bulk operations execute in constant time if their argument is also an enum set.

This class is a member of the Java Collections Framework.

Since:

1.5

See Also:

`EnumMap`

Method Summary

Concrete Methods **Instance Method**

Modifier and Type	Method	Description
`static <E extends Enum<E>> EnumSet<E>`	`allOf(Class<E> elementType)`	Creates an enum set containing all of the elements in the specified element type.
`EnumSet<E>`	`clone()`	Returns a copy of this set.
`static <E extends Enum<E>> EnumSet<E>`	`complementOf(EnumSet<E> s)`	Creates an enum set with the same element type as the specified

		enum set, initially containing all the elements of this type that are *not* contained in the specified set.
`static <E extends Enum<E>> EnumSet<E>`	`copyOf(Collection<E> c)`	Creates an enum set initialized from the specified collection.
`static <E extends Enum<E>> EnumSet<E>`	`copyOf(EnumSet<E> s)`	Creates an enum set with the same element type as the specified enum set, initially containing the same elements (if any).
`static <E extends Enum<E>> EnumSet<E>`	`noneOf(Class<E> elementType)`	Creates an empty enum set with the specified element type.
`static <E extends Enum<E>> EnumSet<E>`	`of(E e)`	Creates an enum set initially containing

		the specified element.
static <E extends Enum<E>> EnumSet<E>	of(E first, E... rest)	Creates an enum set initially containing the specified elements.
static <E extends Enum<E>> EnumSet<E>	of(E e1, E e2)	Creates an enum set initially containing the specified elements.
static <E extends Enum<E>> EnumSet<E>	of(E e1, E e2, E e3)	Creates an enum set initially containing the specified elements.
static <E extends Enum<E>> EnumSet<E>	of(E e1, E e2, E e3, E e4)	Creates an enum set initially containing the specified elements.
static <E extends Enum<E>> EnumSet<E>	of(E e1, E e2, E e3, E e4, E e5)	Creates an enum set initially containing the specified elements.
	range(E from, E to)	

static <E extends Enum<E>> EnumSet<E>	Creates an enum set initially containing all of the elements in the range defined by the two specified endpoints.

Methods inherited from class java.util.AbstractSet

equals, hashCode, removeAll

Methods inherited from class java.util.AbstractCollection

add, addAll, clear, contains, containsAll, isEmpty, iterator, remove, retainAll, size, toArray, toArray, toString

Methods inherited from class java.lang.Object

finalize, getClass, notify, notifyAll, wait, wait, wait

Methods inherited from interface java.util.Set

add, addAll, clear, contains, containsAll, isEmpty, iterator, remove, retainAll, size, spliterator, toArray, toArray

Methods inherited from interface java.util.Collection

parallelStream, removeIf, stream

Methods inherited from interface java.lang.Iterable

forEach

Method Detail

noneOf

```
public static <E extends Enum<E>> EnumSet<E> noneOf(Class<E> elementType)
```

Creates an empty enum set with the specified element type.

Type Parameters:

`E - The class of the elements in the set`

Parameters:

`elementType - the class object of the element type for this enum set`

Returns:

`An empty enum set of the specified type.`

Throws:

`NullPointerException - if elementType is null`

allOf

```
public static <E extends Enum<E>> EnumSet<E> allOf(Class<E> elementType)
```

Creates an enum set containing all of the elements in the specified element type.

Type Parameters:

`E - The class of the elements in the set`

Parameters:

`elementType - the class object of the element type for this enum set`

Returns:

`An enum set containing all the elements in the specified type.`

Throws:

`NullPointerException - if elementType is null`

copyOf

```
public static <E extends Enum<E>> EnumSet<E> copyOf(EnumSet<E> s)
```

Creates an enum set with the same element type as the specified enum set, initially containing the same elements (if any).

Type Parameters:

E - The class of the elements in the set

Parameters:

s - the enum set from which to initialize this enum set

Returns:

A copy of the specified enum set.

Throws:

NullPointerException - if s is null

copyOf

```
public static <E extends Enum<E>> EnumSet<E> copyOf(Collection<E> c)
```

Creates an enum set initialized from the specified collection. If the specified collection is an EnumSet instance, this static factory method behaves identically to copyOf(EnumSet). Otherwise, the specified collection must contain at least one element (in order to determine the new enum set's element type).

Type Parameters:

E - The class of the elements in the collection

Parameters:

c - the collection from which to initialize this enum set

Returns:

An enum set initialized from the given collection.

Throws:

IllegalArgumentException - if c is not an EnumSet instance and contains no elements

NullPointerException - if c is null

complementOf

```
public static <E extends Enum<E>> EnumSet<E> complementOf(EnumSet<E> s)
```

Creates an enum set with the same element type as the specified enum set, initially containing all the elements of this type that are *not* contained in the specified set.

Type Parameters:

E - The class of the elements in the enum set

Parameters:

s - the enum set from whose complement to initialize this enum set

Returns:

The complement of the specified set in this set

Throws:

NullPointerException - if s is null

of

```
public static <E extends Enum<E>> EnumSet<E> of(E e)
```

Creates an enum set initially containing the specified element. Overloadings of this method exist to initialize an enum set with one through five elements. A sixth overloading is provided that uses the varargs feature. This overloading may be used to create an enum set initially containing an arbitrary number of elements, but is likely to run slower than the overloadings that do not use varargs.

Type Parameters:

E - The class of the specified element and of the set

Parameters:

e - the element that this set is to contain initially

Returns:

an enum set initially containing the specified element

Throws:

NullPointerException - if e is null

of

```
public static <E extends Enum<E>> EnumSet<E> of(E e1,
                                                E e2)
```

Creates an enum set initially containing the specified elements. Overloadings of this method exist to initialize an enum set with one through five elements. A sixth overloading is provided that uses the varargs feature. This overloading may be used to create an enum set initially containing an arbitrary number of elements, but is likely to run slower than the overloadings that do not use varargs.

Type Parameters:

E - The class of the parameter elements and of the set

Parameters:

e1 - an element that this set is to contain initially

e2 - another element that this set is to contain initially

Returns:

an enum set initially containing the specified elements

Throws:

NullPointerException - if any parameters are null

of

```
public static <E extends Enum<E>> EnumSet<E> of(E e1,
                                                E e2,
                                                E e3)
```

Creates an enum set initially containing the specified elements. Overloadings of this method exist to initialize an enum set with one through five elements. A sixth overloading is provided that uses the varargs feature. This overloading may be used to create an enum set initially containing an arbitrary number of elements, but is likely to run slower than the overloadings that do not use varargs.

Type Parameters:

E - The class of the parameter elements and of the set

Parameters:

e1 - an element that this set is to contain initially

e2 - another element that this set is to contain initially

e3 - another element that this set is to contain initially

Returns:

an enum set initially containing the specified elements

Throws:

NullPointerException - if any parameters are null

of

```
public static <E extends Enum<E>> EnumSet<E> of(E e1,
                                                E e2,
                                                E e3,
                                                E e4)
```

Creates an enum set initially containing the specified elements. Overloadings of this method exist to initialize an enum set with one through five elements. A sixth overloading is provided that uses the varargs feature. This overloading may be used to create an enum set initially containing an arbitrary number of elements, but is likely to run slower than the overloadings that do not use varargs.

Type Parameters:

E - The class of the parameter elements and of the set

Parameters:

e1 - an element that this set is to contain initially

e2 - another element that this set is to contain initially

e3 - another element that this set is to contain initially

e4 - another element that this set is to contain initially

Returns:

an enum set initially containing the specified elements

Throws:

NullPointerException - if any parameters are null

of

```
public static <E extends Enum<E>> EnumSet<E> of(E e1,
                                                E e2,
                                                E e3,
                                                E e4,
                                                E e5)
```

Creates an enum set initially containing the specified elements. Overloadings of this method exist to initialize an enum set with one through five elements. A sixth overloading is provided that uses the varargs feature. This overloading may be used to create an enum set initially containing an arbitrary number of elements, but is likely to run slower than the overloadings that do not use varargs.

Type Parameters:

E - The class of the parameter elements and of the set

Parameters:

e1 - an element that this set is to contain initially

e2 - another element that this set is to contain initially

e3 - another element that this set is to contain initially

e4 - another element that this set is to contain initially

e5 - another element that this set is to contain initially

Returns:

an enum set initially containing the specified elements

Throws:

NullPointerException - if any parameters are null

of

```
@SafeVarargs
public static <E extends Enum<E>> EnumSet<E> of(E first,
                                                E... rest)
```

Creates an enum set initially containing the specified elements. This factory, whose parameter list uses the varargs feature, may be used to create an enum set initially containing an arbitrary number of elements, but it is likely to run slower than the overloadings that do not use varargs.

Type Parameters:

E - The class of the parameter elements and of the set

Parameters:

first - an element that the set is to contain initially

rest - the remaining elements the set is to contain initially

Returns:

an enum set initially containing the specified elements

Throws:

NullPointerException - if any of the specified elements are null, or if rest is null

range

```
public static <E extends Enum<E>> EnumSet<E> range(E from,
                                                   E to)
```

Creates an enum set initially containing all of the elements in the range defined by the two specified endpoints. The returned set will contain the endpoints themselves, which may be identical but must not be out of order.

Type Parameters:

E - The class of the parameter elements and of the set

Parameters:

from - the first element in the range

to - the last element in the range

Returns:

an enum set initially containing all of the elements in the range defined by the two specified endpoints

Throws:

NullPointerException - if from or to are null

IllegalArgumentException - if from.compareTo(to) > 0

clone

```
public EnumSet<E> clone()
```

Returns a copy of this set.

Overrides:

`clone` in class `Object`

Returns:

a copy of this set

See Also:

`Cloneable`

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.